
Cout de calcul de la recherche

Du decrochage self-play au lot de forme fixe dans LapZero-Chess

Rapport d'ingenierie

Mathieu Leclercq
19 septembre 2026

Version 1.0
Dernier commit cite : fb2ca71 (branche main)

Ce document raconte comment la recherche MCTS de LapZero-Chess, quatre fois plus lente par position que le self-play a reseau et materiel identiques, a retrouve le niveau du self-play en corrigeant une seule chose : la forme du lot envoyee a ONNX Runtime. Il decrit la demarche de mesure qui a isole la cause, la correction, les pistes essayees et refusees, et ce que les chiffres disent du plafond restant. Toutes les valeurs proviennent de campagnes conservees dans `out/multicore/`; les commits cites sont dans l'historique Git du depot.

Table des matières

Résumé	3
1 Introduction : deux chemins, un même réseau	4
1.1 L'observation	4
1.2 La question	4
1.3 La méthode et les contributions	5
2 Contexte : ce que le projet savait déjà	5
2.1 Batching et virtual loss	5
2.2 Vagues multicœur	5
2.3 Répartition par phase	6
2.4 Le modèle $F + M \cdot N$	6
2.5 L'étude de coût et ses hypothèses	6
2.6 Le régime réel du bot	6
3 Conception : mesurer hors arbre, puis corriger	6
3.1 Mesurer l'évaluateur hors arbre	7
3.2 La découverte : la forme du lot	7
3.3 La correction : lot de forme fixe	8
3.4 Les alternatives écartées	9
3.5 La barrière qualité	9
4 Implémentation	9
4.1 Vue d'ensemble	9
4.2 Le banc évaluateur	9
4.3 Le lot fixe dans les vagues	10
4.4 Le chemin mono	10
4.5 L'activation UCI	11
4.6 La divergence paramétrable	11
4.7 Le cas chaud	11
4.8 Les commits	11
5 Validation	11
5.1 La stratégie	11
5.2 Le lot fixe	12
5.3 La divergence	12
5.4 La barrière qualité	12
5.5 Les suites et le smoke	12
6 Mesures	13
6.1 Protocole général	13
6.2 La courbe de l'évaluateur	13
6.3 Le lot fixe sur la recherche	13
6.4 Le balayage de divergence	14
6.5 FP16	15
6.6 Le cas chaud	15
6.7 Synthèse	16

7	Discussion	16
7.1	Ce que le gain signifie	16
7.2	Le plafond restant	16
7.3	Les leçons	17
7.4	Les limites	17
7.5	La production continue	17
8	Conclusion	18
A	Chronologie des commits	18
B	Inventaire des fichiers	19
C	Reproduction des mesures	20
C.1	Compilation et suites	20
C.2	Banc évaluateur	20
C.3	Lot fixe sur la recherche	20
C.4	Balayage de divergence et qualité	20
C.5	FP16 et cas chaud	21
C.6	Figures et rapport	21
D	Glossaire	21

Résumé

Ce rapport raconte un diagnostic et sa correction. Après le chantier multicœur, la recherche UCI de LapZero-Chess tournait entre 500 et 850 simulations par seconde selon la position, alors que le même réseau, sur le même GPU, était évalué en 0,38 ms par position par le banc self-play, soit environ 2 600 évaluations par seconde. Un facteur quatre séparait deux chemins qui partagent pourtant le même évaluateur ONNX. Le moteur n'avait aucune explication, et trois hypothèses se disputaient la cause : la plomberie de l'appel, les allocations par appel, ou l'occupation du GPU.

La réponse est venue d'un banc hors arbre : en lots de forme *fixe*, l'appel coûte 2,7 à 5,1 ms selon la taille ; en lots *alternés* de 1 à 8, il coûte 13,4 ms quel que soit le lot. ONNX Runtime re-planifie son graphe et sa mémoire dès que la forme du tenseur change, et la recherche alternait précisément une racine à 1 et des vagues partielles, alors que le self-play évaluait toujours 8. La correction tient en une idée : *padder* chaque lot à la taille demandée en dupliquant le dernier tenseur et ignorer les sorties dupliquées. La recherche est sémantiquement inchangée, et le débit passe à 1 500–2 100 simulations par seconde, soit un facteur 2,0 à 3,2 selon la position, avec un p95 de latence divisé par deux à trois.

Le même chantier a évalué deux autres pistes. La conversion FP16 est plus lente de 18 à 28 % et a été rejetée sans campagne qualité. Le balayage de divergence gagne 13 à 25 % de débit mais dégrade la qualité de 1,8 à 2,4 points sur 500 puzzles : les réglages par défaut sont conservés. Enfin, la mesure du cas chaud a corrigé une croyance du projet : un succès de table n'évite pas l'inférence, car il matérialise les enfants et la descente continue vers une feuille réseau. La réserve Amdahl des rapports précédents est sans objet.

L'essentiel en dix points.

1. Le décrochage entre self-play (0,38 ms par position) et recherche (1,4 à 2 ms) venait du *changement de forme du lot* à chaque appel ONNX Runtime, pas du réseau ni de l'occupation GPU.
2. En lots alternés 1 à 8, *session->Run* coûte 13,4 ms contre 2,7 à 3,7 ms à forme fixe. A/B interleaved : 13,0 ms contre 3,3 ms, facteur quatre.
3. La correction *padde* les vagues à *batch_size* en dupliquant le dernier tenseur et n'utilise que les sorties des positions réelles. Les positions réelles gardent leurs indices, donc leurs sorties sont identiques : la recherche est inchangée.
4. Gains mesurés en A/B interleaved, 700 simulations : $\times 2,00$ en ouverture, $\times 3,18$ en milieu, $\times 2,55$ en finale pour 8 workers ; $\times 2,0$ à $\times 3,3$ pour le chemin mono.
5. Le p95 de latence est divisé par 2 à 3 (rapports 0,30 à 0,56), pas dégradé.
6. Le bot UCI active le lot fixe (`MCTS_FIXED_BATCH = True`) et passe de 500–850 à 1 500–2 100 simulations par seconde, validé par un smoke en sous-processus.
7. Le softmax C++ sur 4 672 sorties ne pèse que 0,2 à 2 % de l'appel : ce n'était pas un levier.
8. FP16 rejeté : 18 à 28 % plus lent en `keep_io_types`.
9. Divergence : `vloss 2` et `vloss 2 + FPU 0,45` gagnent 13 à 25 % de débit mais coûtent 1,8 à 2,4 points de résolution sur 500 puzzles ; les défauts sont conservés.
10. Cas chaud : le débit tient à 2 000–2 200 simulations par seconde et les appels réseau restent à 700 pour 700 simulations, même quand les succès de table montent de 8 à 81 %.

Mots-clés : MCTS, ONNX Runtime, forme de lot, performance, C++17, GPU, virtual loss, self-play, AlphaZero.

Comment lire ce document

Le chapitre 1 pose l'énigme et la méthode. Le chapitre 2 décrit l'état du moteur et les hypothèses en présence. Le chapitre 3 est le cœur : la conception du banc, la découverte, la correction et les alternatives refusées. Le chapitre 4 raconte l'implémentation et ses pièges. Le chapitre 5 décrit la validation. Le chapitre 6 rassemble les campagnes de mesure, avec leurs courbes. Le chapitre 7 discute la portée des résultats et le plafond restant. Le chapitre 8 conclut. Les annexes donnent la chronologie des commits, l'inventaire des fichiers, les commandes de reproduction et un glossaire.

Les encadrés bleus résument, les encadrés ambre signalent les pièges vécus, et les encadrés gris décrivent les protocoles de mesure. Les valeurs numériques proviennent des fichiers `out/multicore/` conservés hors du dépôt.

1 Introduction : deux chemins, un même réseau

1.1 L'observation

À la fin du chantier multicœur, la recherche UCI de LapZero-Chess tournait entre 500 et 850 simulations par seconde selon la position et le nombre de workers. Le réseau était un ResNet de 10 blocs et 128 filtres, exporté en ONNX, exécuté sur un GPU RTX PRO 2000 Blackwell portable avec le provider CUDA d'ONNX Runtime. Le banc self-play, lui, mesurait 3,05 ms pour un lot de 8 positions, soit 0,38 ms par position et environ 2 600 évaluations par seconde, avec exactement le même modèle et le même GPU.

Le tableau 1 résume l'écart : par position, la recherche payait 1,4 à 2,9 ms là où le self-play payait 0,38 ms, un facteur quatre à huit.

TABLEAU 1 – Coût par position avant le chantier. Recherche : phase évaluateur de 700 simulations, chemin `step_analysis`, lot 8, workers 8. Self-play : banc partagé, 8 plateaux, 100 vagues.

Chemin	ms par position	sims/s	Source
Self-play, lot 8 fixe	0,38	≈ 2 600	<code>selfplay-after-gpu.json</code>
Recherche, ouverture	1,19	847	<code>after.json</code>
Recherche, milieu	2,87	349	<code>after.json</code>
Recherche, finale	1,77	565	<code>after.json</code>

1.2 La question

L'évaluateur est le même objet `ONNXEvaluator` dans les deux cas. La différence ne pouvait donc pas venir du réseau lui-même. Trois familles d'hypothèses restaient :

1. **La plomberie de l'appel.** La recherche reconstruit un vecteur d'entrée par vague, redimensionne les sorties et appelle le même `evaluate_batch`, mais avec des tailles de lot variables et des buffers différents à chaque appel. Le self-play appelle en boucle serrée avec un buffer fixe.
2. **Les allocations.** Chaque vague alloue un vecteur d'entrée d'environ 200 kio et laisse `evaluate_batch` redimensionner les sorties. Le self-play réutilise tout.
3. **L'occupation GPU.** Le réseau est minuscule : cartes 8×8, 128 filtres, environ 0,4 GFLOP par position. Le GPU pouvait être sous-occupé, et le coût réel pouvait venir des noyaux et non des données.

Les trois hypothèses ne s'excluent pas, et l'étude de coût 2026-09-19-cout-calcul-cpu-gpu.md les avait listées sans pouvoir trancher, faute de mesure qui sépare l'appel de l'arbre.

1.3 La méthode et les contributions

La démarche retenue est expérimentale et hiérarchisée : mesurer d'abord l'évaluateur *hors arbre*, puis ne corriger que la cause démontrée, et refuser par la mesure toute piste qui ne passe pas la barrière qualité.

Les contributions de ce chantier

- un banc C++ evaluator_batch_bench qui mesure la courbe du lot hors arbre, GPU et CPU, avec un mode « lots alternés » qui reproduit la recherche, et des chronomètres internes séparant session->Run du softmax;
- la découverte et la quantification de la re-planification d'ONNX Runtime au changement de forme : facteur quatre sur l'appel;
- le lot de forme fixe, appliqué aux vagues multicœur et au chemin mono, sémantiquement neutre et activé dans le bot UCI;
- l'évaluation de deux pistes concurrentes, FP16 et réglages de divergence, toutes deux refusées sur preuves;
- la mesure du cas chaud, qui corrige une réserve des rapports précédents sur la loi d'Amdahl;
- un verdict chiffré sur la production continue et le plafond restant.

Le reste du rapport suit l'ordre : contexte et hypothèses (chapitre 2), conception et découverte (chapitre 3), implémentation (chapitre 4), validation (chapitre 5), mesures (chapitre 6), discussion (chapitre 7), conclusion (chapitre 8).

2 Contexte : ce que le projet savait déjà

2.1 Batching et virtual loss

La recherche regroupe déjà plusieurs feuilles dans une seule inférence. Chaque descente pose un *virtual loss* de type Leela Chess Zero : un compteur `n_in_flight` incrémenté le long du chemin, qui n'entre que dans le dénominateur du terme d'exploration et laisse Q intact. Le noyau mono-worker collecte jusqu'à `batch_size` feuilles, s'arrête au premier nœud déjà réservé, puis appelle l'évaluateur une fois. Le balayage de batching du 14 septembre 2026 avait fixé le lot de production à 8, meilleur compromis entre débit et qualité.

2.2 Vagues multicœur

Le chantier multicœur a remplacé la collecte mono-cœur par des collecteurs persistants travaillant sur un arbre partagé, avec réservation atomique des feuilles, table de transposition par snapshots, garde RAI et budget exact. Le rapport de résultats 2026-09-16-multicore-waves-results.md mesure :

- des gains de +6 à +22 % de débit selon la position entre 1 et 8 workers, avec un plafond dès deux workers;
- un p95 de latence sous +5 % partout, et même amélioré en finale;
- une qualité non inférieure sur 2 500 puzzles (delta -0,28 point, IC95 [-0,8 ; +0,24]);
- l'activation de `MCTS_WORKER_COUNT = 8` dans `uci.py`.

2.3 Répartition par phase

Les chronométrages internes (phases . json, 700 simulations, lot 8) montrent une domination écrasante de l'évaluateur :

TABLEAU 2 – Part de l'évaluateur dans le temps mur, 700 simulations.

Position	Total (ms)	Évaluateur (ms)	Part	CPU + barrière
Ouverture, w1	923,8	903,2	97,8 %	20,6 ms
Milieu, w1	2 063,5	2 038,1	98,8 %	25,4 ms
Finale, w1	1 447,5	1 426,4	98,5 %	21,1 ms
Finale, w8	1 300,4	1 264,0	97,2 %	36,4 ms
Milieu, w8	2 335,3	2 280,8	97,7 %	54,5 ms
Ouverture, w8	853,3	824,2	96,6 %	29,1 ms

Le travail CPU de recherche, coups compris, vaut 29 à 36 microsecondes par simulation, soit 1 à 1,5 % du temps. La barrière des vagues coûte 1,5 à 2,3 %. Autrement dit, tout gain durable ne peut venir que de l'appel réseau.

2.4 Le modèle $F + M \cdot N$

Le balayage de lots du 14 septembre 2026, mesuré dans une seule session, montre que le débit plafonne dès le lot 8 : demander 64 positions n'en fait évaluer que 5 à 10, parce que l'arbre ne fournit pas plus de feuilles utiles. En ajustant $T(N) = F + M \cdot N$ sur le lot 1 et le lot 8, on trouve un coût fixe F d'environ 1 à 2 ms et un coût par position M de 0,9 à 1,9 ms selon la position, d'où un plafond du batching seul de $\times 1,6$ à $\times 3,4$.

Ce modèle décrivait bien les gains observés, mais il laissait une incohérence : le self-play, avec un lot plus grand mais surtout *fixe*, obtenait 0,38 ms par position, incompréhensible dans un modèle où M devrait être une propriété du réseau.

2.5 L'étude de coût et ses hypothèses

L'étude 2026-09-19-cout-calcul-cpu-gpu.md avait agrégé les mesures et comparé le moteur à Leela Chess Zero. Elle estimait l'écart à un facteur 15 à 200 selon quatre axes : taille de lot ($\times 3$ à 8), exécution asynchrone ($\times 1,2$ à 1,5), backend réseau ($\times 2$ à 4) et cache (à 4). Elle notait surtout que le premier facteur, la plomberie de l'appel, était déjà prouvé par le banc self-play, et que le lot d'un seul arbre plafonnant vers 7, le gain total réaliste était de $\times 3$ à $\times 6$, pas $\times 30$.

C'est cette piste, et non le réseau, que le plan 2026-09-19-cout-calcul-optimisations.md a mise en premier, avec un banc hors arbre comme étape préalable.

2.6 Le régime réel du bot

Le bot n'appelle pas `step_analysis` une fois pour 700 simulations : il découpe en tranches de `BATCH_SIZE = 64` pour garder la main sur l'horloge, avec arbre et pool persistants. Le rapport multicœur mesurait déjà que le gain des workers dépendait de la tranche : +5 à +27 % à 64 simulations, mais -19 % à 8, où les arbres naissants provoquent environ 970 collisions pour 800 simulations. Toutes les campagnes de ce rapport mesurent donc aussi aux tranches réelles, en plus des appels longs.

3 Conception : mesurer hors arbre, puis corriger

3.1 Mesurer l'évaluateur hors arbre

L'arbre cache la cause : quand la recherche ralentit, on ne sait pas si c'est la collecte, la forme du lot, le buffer ou le GPU. Le premier livrable du chantier est donc un banc qui appelle `evaluate_batch` directement, sur des tenseurs aléatoires, sans arbre.

Protocole du banc évaluateur

- lots fixes balayés de 1 à 256, avec échauffement, 30 répétitions et 20 appels par mesure ; les lots 3, 5, 6, 7, 12, 24, 48 et 96 sont mesurés dans une seconde campagne ;
- un mode -variable qui alterne les lots 1 à 8 à chaque appel, comme le fait la recherche, et accumule par taille ;
- un mode -fake qui valide le harnais avec `ControlledEvaluator`, sans modèle ;
- deux chronomètres internes optionnels, désactivés par défaut en production, qui séparent `session->Run` du softmax C++.

Le banc sort un JSON par mesure, agrégé ensuite en médianes. Le modèle et le GPU sont ceux de production, et le provider CUDA est le même.

3.2 La découverte : la forme du lot

La figure 1 et le tableau 3 montrent le résultat. En lots fixes, le coût par position décroît de 9,75 ms (lot 1) à 0,10 ms (lot 128). En lots alternés 1 à 8, chaque appel coûte 13,5 ms, dont 13,4 ms de `session->Run`, *quel que soit le lot*. Le contraste est brutal : la recherche ne payait pas le réseau, elle payait le changement de forme.

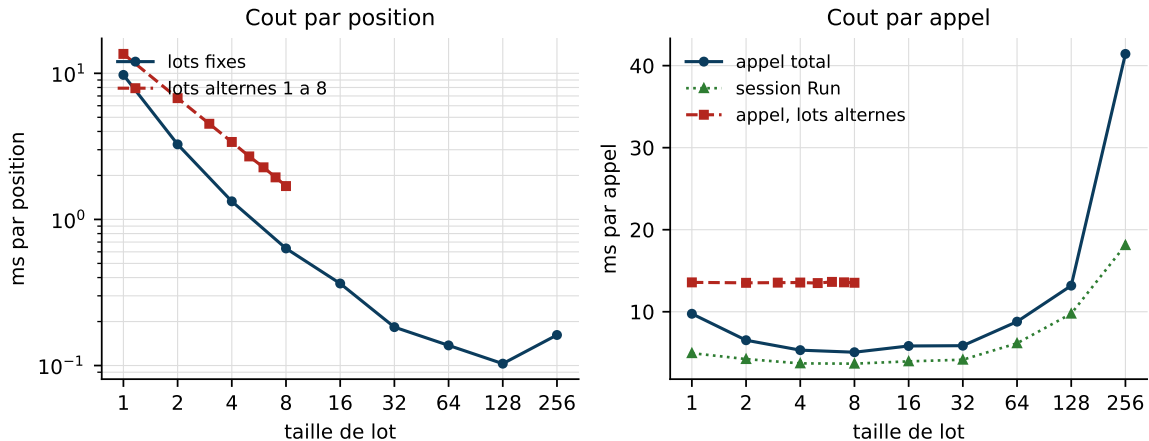


FIGURE 1 – Coût de l'évaluateur hors arbre, GPU. À gauche, le coût par position en lots fixes (bleu) et en lots alternés (rouge). À droite, le coût par appel, avec la part `session->Run` en pointillés verts.

Les lots partiels mesurés isolément (3, 5, 6, 7) coûtent tous environ 3,4 ms, *moins* que le lot 8 de la première campagne. Autrement dit, aucune forme prise seule n'est chère ; c'est leur alternance qui l'est. Un A/B interleaved serré confirme : 13,03 et 13,04 ms en alternance contre 2,87 et 3,65 ms à forme fixe 8, soit un facteur quatre.

Le piège : accuser le réseau ou les allocations

Les allocations par vague et le softmax C++ étaient les coupables désignés. Les mesures les innocentent : le softmax vaut 0,02 à 0,10 ms par appel, soit 0,2 à 2 %, et retirer les allocations par appel ne changerait pas l'ordre de grandeur. La cause était le contrat

TABLEAU 3 – Courbe du lot, GPU, lots fixes (médianes).

Lot	ms par appel	ms par position	run ONNX (ms)
1	9,75	9,75	4,95
2	6,53	3,26	4,23
4	5,32	1,33	3,71
8	5,07	0,63	3,67
16	5,83	0,36	3,95
32	5,86	0,18	4,15
64	8,79	0,14	6,16
128	13,18	0,10	9,77
256	41,43	0,16	18,12

implicite d'ONNX Runtime : une forme de tenseur stable est un pattern mémoire réutilisable, une forme qui change est un graphe à re-planifier.

3.3 La correction : lot de forme fixe

La correction consiste à ne plus jamais changer la forme du lot en cours de recherche. Chaque vague collecte un nombre variable de feuilles réseau ; le coordinateur duplique le dernier tenseur jusqu'à batch_size, appelle l'évaluateur avec la forme fixe, puis n'utilise que les sorties des positions réelles.

```

1  const int eval_batch = m_fixed_batch ? batch_size
2                                : static_cast<int>(network_count);
3  if (m_fixed_batch && network_count > 0) {
4      eval_batch = batch_size;
5      const std::size_t target = batch_size * TENSOR_SIZE;
6      if (batch_input.size() < target) {
7          batch_input.reserve(target);
8          const std::vector<float> dernier(
9              batch_input.end() - TENSOR_SIZE, batch_input.end());
10         while (batch_input.size() < target) {
11             batch_input.insert(batch_input.end(),
12                               dernier.begin(), dernier.end());
13         }
14     }
15 }
```

Listing 1 – Padding d'une vague (extrait de mcts_wave.cpp).

Trois choix de conception méritent d'être explicités.

Pourquoi dupliquer le dernier tenseur plutôt que des zéros. Une rangée dupliquée coûte le même temps GPU qu'une rangée réelle, mais elle ne demande aucun remplissage supplémentaire et le tenseur est déjà contigu dans le buffer. Le contenu n'a aucune importance puisque la sortie est ignorée.

Pourquoi laisser la racine au lot 1. L'expansion de racine est un appel unique par coup, avec un seul tenseur. La padder à 8 gaspillerait sept évaluations par coup pour rien. Il reste donc deux formes dans une recherche, 1 puis 8 : la première vague après la racine paie une re-planification, négligeable sur un coup entier.

Pourquoi la recherche est sémantiquement inchangée. Les positions réelles occupent les indices 0 à network_count-1 du lot, avant et après le padding ; la sortie d'une rangée ne dépend pas des autres rangées, le softmax étant calculé par position. Chaque feuille réelle reçoit donc exactement la policy et la valeur qu'elle aurait eues sans padding. Seules changent la taille du tenseur d'entrée et les rangées supplémentaires, dont les sorties sont jetées.

3.4 Les alternatives écartées

TABLEAU 4 – Pistes examinées et verdict.

Piste	Raison	Verdict
Buffers persistants	Le facteur dominant est la forme, pas l'allocation ; les vagues allouent environ 200 kio par appel, soit moins de 1 %	non retenue
IO binding, CUDA graph	Complexité de plomberie sans cause démontrée au-delà de la forme	non retenue
Softmax fusionné ou restreint	0,2 à 2 % de l'appel	non retenue
FP16	18 à 28 % plus lent en <code>keep_io_types</code> (chapitre 6)	rejetée
Divergence (vloss, FPU)	+13 à +25 % de débit mais -1,8 à -2,4 points de résolution	rejetée
Production continue	Gain potentiel $\times 1,3$ à $1,7$, hors périmètre de ce plan	plan séparé

3.5 La barrière qualité

Le lot fixe ne change pas la recherche, il n'exige donc aucune validation qualité. En revanche, toute modification qui touche la sélection, le virtual loss ou la précision du réseau en exige une : c'est la règle posée par le plan. Les candidats de divergence ont été testés sur 500 puzzles avec le comparateur apparié du chantier précédent (`multicore_comparison.py`), qui refuse doublons, lignes manquantes, erreurs et métadonnées incompatibles, et calcule un intervalle bootstrap apparié à 20 000 tirages.

4 Implémentation

4.1 Vue d'ensemble

Le chantier suit le plan `2026-09-19-cout-calcul-optimisations.md` en six tâches : banc hors arbre, buffers, lot fixe, divergence, FP16, cas chaud, puis mesures et décision. L'ordre a été adapté en cours de route : la découverte du facteur quatre a fait passer le lot fixe avant les buffers, devenus marginaux.

TABLEAU 5 – Tâches et état.

Tâche	État	Livrable
1. Banc évaluateur	faite	<code>evaluator_batch_bench</code> , courbes du lot
2. Buffers persistants	non retenue	mesure : la forme domine
3. Lot fixe	faite	vagues et mono, activé dans l'UCI
4. Divergence paramétrable	faite, candidats rejetés	SearchTuning, balayage
5. FP16	rejetée par la mesure	conversion testée, non adoptée
6. Cas chaud	faite	<code>hot_tree_bench.py</code>
7. Mesures et décision	faite	<code>2026-09-19-cout-calcul-resultats.md</code>

4.2 Le banc évaluateur

Le nouveau binaire `evaluator_batch_bench` reprend la structure de `selfplay_phase_bench` : options parsées à la main, sortie JSON sur la sortie standard, cible CMake avec copie des DLL ONNX et des providers. Il accepte `-fake` pour valider le harnais sans modèle, `-batches`, `-repetitions`, `-rounds`, `-variable` et `-out-json`.

Les chronomètres internes sont ajoutés à `ONNXEvaluator`, désactivés par défaut pour que la production ne paie rien :

```

1 const auto run_start = m_timing_enabled
2   ? std::chrono::steady_clock::now()
3   : std::chrono::steady_clock::time_point{};
4 try {
5     output_tensors = session->Run(Ort::RunOptions{ nullptr },
6                                   input_names, &input_ort, 1,
7                                   output_names, 2);
8 }
9 ...
10 if (m_timing_enabled) {
11     m_last_timing.run_ns = ...;
12 }

```

Listing 2 – Chronométrage interne optionnel (`onnx_evaluator.cpp`).

Un `CTest evaluator_batch_bench_smoke` lance le mode faux sur les lots 1, 4 et 8 et vérifie par expression régulière la présence du lot 8, de la deuxième répétition et des deux appels par mesure.

4.3 Le lot fixe dans les vagues

MCTS gagne un drapeau par session, `m_fixed_batch`, faux par défaut, et deux accesseurs `set_fixed_batch` / `fixed_batch` protégés par le verrou de session. Le coordinateur des vagues padde le lot comme montré au chapitre 3, puis dimensionne sa vérification sur `eval_batch` au lieu de `network_count`. Les compteurs gardent leur sens : `nn_calls` compte les positions réelles, `nn_batches` les appels physiques.

```

1 m_nn_calls.fetch_add(network_count, std::memory_order_relaxed);
2 m_nn_batches.fetch_add(1, std::memory_order_relaxed);
3 {
4     PhaseTimer evaluator_timer(timing, SearchPhase::Evaluator);
5     m_evaluator->evaluate_batch(batch_input, policies, values, eval_batch);
6 }
7 const std::size_t expected_policy =
8     static_cast<std::size_t>(eval_batch) * 4672;
9 if (policies.size() != expected_policy
10     || values.size() != static_cast<std::size_t>(eval_batch)) {
11     throw std::runtime_error("evalateur : dimensions de sortie invalides");
12 }

```

Listing 3 – Compteurs et vérification de sortie dans les vagues.

4.4 Le chemin mono

Le chemin mono-worker du noyau batché souffre du même mal : sa boucle ferme le lot au premier nœud déjà collecté et évalue donc des lots de tailles variables. Le même padding y est appliqué, avec une vérification de dimensions ajoutée au passage, ce qui accélère aussi le banc de puzzles et les tournois qui appellent `mcts_search`.

Le binding oublié

La première campagne `-fixed-batch` a échoué immédiatement : le drapeau existait en C++ et dans le banc Python, mais `set_fixed_batch` n'était pas exposé par `pybind11`, d'où une `AttributeError` côté Python. Le binding a été ajouté avec `set_tuning` et `get_tuning`, puis l'extension reconstruite. Les tests Python utilisant des faux MCTS ont été complétés en conséquence.

4.5 L'activation UCI

Le bot active le drapeau à la construction, uniquement sur le chemin de production, pour ne pas casser les tests qui injectent un faux MCTS :

```

1 MCTS_FIXED_BATCH = True
2 ...
3 self.mcts = (mcts if mcts is not None
4               else chess_engine.MCTS(self.evaluator, tt_size=4_000_000))
5 if mcts is None:
6     self.mcts.set_fixed_batch(MCTS_FIXED_BATCH)
7 ...
8 f"uci.py pret : modele {Path(MODEL_PATH).name}, "
9 f"{self.provider}, lot {MCTS_BATCH_SIZE}, "
10 f"lot fixe {MCTS_FIXED_BATCH}, "
11 f"workers {MCTS_WORKER_COUNT}, TT 4000000"

```

Listing 4 – uci.py : activation et journal.

4.6 La divergence paramétrable

Pour pouvoir balayer la divergence sans recompiler, MCTS expose un petit contrat SearchTuning (amplitude du virtual loss, coefficient FPU, facteur de tentatives de collision), validé à l'entrée et de valeurs par défaut inchangées. La garde de réservation prend désormais un nombre d'unités par descente, et les deux collecteurs lisent le coefficient FPU dans les réglages. Les bancs gagnent -virtual-loss, -fpu, -collision-attempts et -slices, ce dernier découpant une recherche en tranches de N simulations pour mesurer le régime réel du bot.

4.7 Le cas chaud

hot_tree_bench.py joue une séquence de coups avec un MCTS persistant, en tranches, et applique update_root après chaque coup comme le bot. Il rapporte par coup le débit, les appels réseau, les succès de table et la fraction de simulations sans inférence, puis résume par position. La logique d'agrégation est pure et testée sans moteur.

4.8 Les commits

Commit	Objet
4161cd5	banc évaluateur et timings internes
29f9458	lot fixe dans les vagues
74122ac	activation dans l'UCI
6606d97	lot fixe dans le chemin mono
4461d80	cause du décrochage consignée
3515f00	divergence paramétrable
0c41d6f	balayage et rejet qualité
aed5f1a	banc du cas chaud
84bae7e	résultats, FP16 rejeté, cas chaud
fb2ca71	clôture du plan

5 Validation

5.1 La stratégie

Trois filets indépendants protègent ce chantier :

1. des tests déterministes avec faux réseau, qui vérifient que le padding ne change ni le budget, ni les invariants, ni la forme des appels;

2. des campagnes A/B interleaved sur le moteur réel, même session, ordre alterné, pour séparer l'effet mesuré de la dérive thermique et d'horloge du GPU, qui atteint 15 à 30 % entre sessions ;
3. une barrière qualité par palette de 500 puzzles pour toute modification de la recherche.

5.2 Le lot fixe

Un test C++ vérifie que, lot fixe activé, une recherche multicœur de 17 simulations produit des appels tous de taille 8 après la racine (qui reste à 1), que le budget de visites est exact, que la session revient au repos et que le plateau d'entrée est intact. Un second test fait de même sur le chemin mono. Un test Python vérifie la transmission du drapeau depuis le banc, et un test UCI vérifie que le constructeur de production active bien `MCTS_FIXED_BATCH`.

Pourquoi aucune campagne qualité n'est exigée ici

Le padding ne modifie que les rangées supplémentaires du tenseur. Une rangée dupliquée reçoit une sortie qui est jetée ; les rangées réelles gardent leur index et leur sortie. La recherche produit donc les mêmes arbres, à l'ordonnancement près des collecteurs parallèles, qui est déjà non déterministe. C'est un changement de performance pure, pas un changement d'algorithme.

5.3 La divergence

Les réglages par défaut sont verrouillés par un test qui compare `get_tuning()` aux valeurs historiques (1, 0,30, 4) et vérifie qu'une amplitude nulle est refusée. Deux tests Python vérifient que les options des bancs transmettent bien le réglage au moteur et que `-slices` découpe correctement une recherche de 700 simulations en tranches de 64.

5.4 La barrière qualité

Le protocole de comparaison reprend celui du chantier multicœur : un sous-échantillon régulier et stratifié de 500 puzzles, un MCTS neuf par puzzle, le même modèle et le même matériel pour la référence et le candidat, et une comparaison appariée stricte. Les verdicts suivent la même marge : non-infériorité si la borne basse de l'IC95 reste au-dessus de -1 point, régression si la borne haute passe dessous, indéterminé entre les deux.

Deux candidats ont été mesurés contre les défauts, lot fixe des deux côtés : `vloss 2 + FPU 0,45` (delta $-1,8$ point, 21 pertes contre 12 gains) et `vloss 2 seul` (delta $-2,4$ point, 27 pertes contre 15 gains). Les deux sont refusés, et les défauts restent en place.

5.5 Les suites et le smoke

TABLEAU 6 – Vérifications finales.

Vérification	Contenu	Résultat
CTest	15 cibles, dont le smoke du banc évaluateur, les deux tests de padding et le stress multicœur	15/15
pytest	261 tests, dont la propagation du lot fixe, la divergence, le comparateur et le cas chaud	261/261
Smoke UCI	<code>uci</code> , <code>isready</code> , <code>position startpos</code> , <code>go nodes 200</code> , <code>stop</code> , quit en sous-processus	<code>bestmove d2d4</code> , sortie propre

Le smoke UCI confirme aussi le journal de démarrage : `lot 8`, `lot fixe True`, `workers 8`, `TT 4000000`.

6 Mesures

6.1 Protocole général

Toutes les comparaisons de débit sont **interleaved** : les configurations comparées alternent dans une même session, sur la même machine, et les médianes sont calculées par position et par chemin. La dérive d'horloge et thermique du GPU atteint 15 à 30 % entre sessions ; seuls les rapports mesurés dans une session sont exploités. Sauf mention contraire : 700 simulations, lot 8, pool de workers chaud, trois positions de référence (ouverture, milieu, finale), politique de table h0, table 8192.

6.2 La courbe de l'évaluateur

La figure 1 du chapitre 3 montre les deux régimes. En lots fixes, le coût par position décroît de 9,75 ms à 0,10 ms entre les lots 1 et 128, puis remonte à 0,16 ms au lot 256 : la courbe est celle d'un coût fixe par appel amorti par la taille du lot, exactement le modèle $F + M \cdot N$. Les lots partiels mesurés isolément (3, 5, 6, 7) coûtent 3,4 à 3,5 ms, et les lots 12, 24, 48 et 96 coûtent 3,6 à 7,9 ms.

La part du softmax C++ par appel vaut 0,02 ms au lot 1, 0,10 ms au lot 8, 0,39 ms au lot 32 et 6,27 ms au lot 256, soit 0,2 à 15 % selon le lot. Au lot de production, il est négligeable.

TABLEAU 7 – Lots alternés contre lots fixes, médianes par appel (ms).

Lot	Fixe	Alterné 1 à 8	Rapport
1	9,75	13,58	×1,39
2	6,53	13,53	×2,07
4	5,32	13,56	×2,55
8	5,07	13,53	×2,67
A/B interleaved, lot 8	2,87 et 3,65	13,03 et 13,04	×4

6.3 Le lot fixe sur la recherche

La figure 2 donne les gains mesurés en A/B interleaved sur le moteur réel, et le tableau 8 les valeurs.

Effet du lot de forme fixe, 700 simulations, step_analysis, round 1

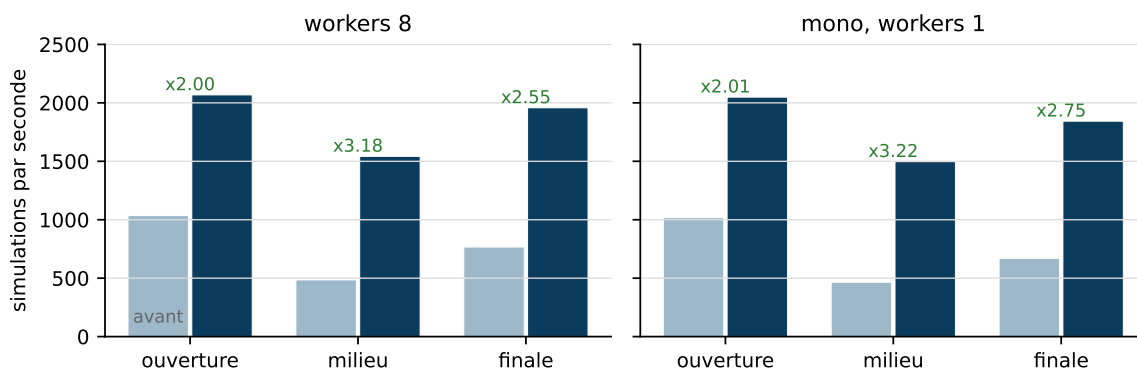
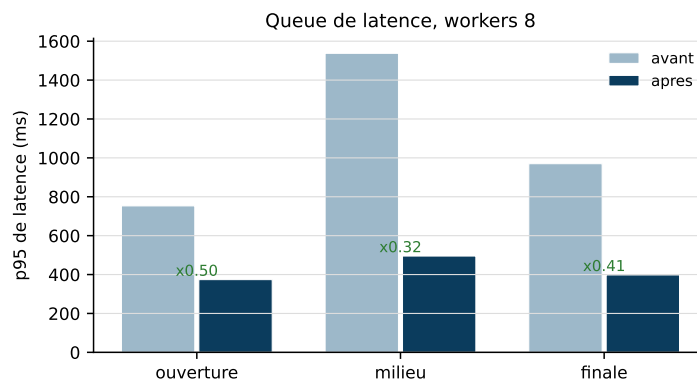


FIGURE 2 – Débit avant et après le lot fixe, 700 simulations, step_analysis, pool chaud. À gauche, 8 workers. À droite, le chemin mono à 1 worker.

TABLEAU 8 – Débit en simulations par seconde, médianes du premier round (11 à 12 observations chaudes par configuration).

Position	8 workers		1 worker	
	Avant	Après	Avant	Après
Ouverture	1 035	2 068 (×2,00)	1 018	2 049 (×2,01)
Milieu	485	1 542 (×3,18)	465	1 497 (×3,22)
Finale	767	1 958 (×2,55)	670	1 843 (×2,75)

Les deux rounds donnent les mêmes rapports à $\pm 0,1$ près. La figure 3 montre la queue de latence : le p95 s'améliore de 2 à 3 fois, au lieu de se dégrader. C'est le résultat attendu d'un appel plus court, mais il fallait le vérifier, le critère d'acceptation du chantier multicœur interdisant toute hausse supérieure à 5 %.

**FIGURE 3** – p95 de latence par recherche de 700 simulations, avant et après, 8 workers.

6.4 Le balayage de divergence

Le balayage croise amplitude de virtual loss, coefficient FPU et budget de tentatives, aux tranches réelles du bot (64 simulations), workers 8, lot fixe, 6 observations par position.

TABLEAU 9 – Balayage de divergence, simulations par seconde (médianes).

Configuration	Ouverture	Milieu	Finale
Défauts (vloss 1, FPU 0,30, tentatives 4)	1 902	1 458	1 749
vloss 2	2 161 (+13,6 %)	1 649 (+13,1 %)	1 973 (+12,8 %)
vloss 3	2 029 (+6,7 %)	1 710 (+17,3 %)	2 108 (+20,6 %)
FPU 0,45	1 950 (+2,5 %)	1 546 (+6,0 %)	1 821 (+4,1 %)
vloss 2 + FPU 0,45	2 093 (+10,0 %)	1 730 (+18,7 %)	2 187 (+25,0 %)
Tentatives ×8	2 008 (+5,6 %)	1 478 (+1,4 %)	1 862 (+6,4 %)

Les gains de débit étaient réels, et le remplissage montait avec l'amplitude du virtual loss. Mais la barrière qualité les a arrêtés :

Les deux candidats sont refusés et les défauts restent. La leçon est nette : *plus de divergence élargit la recherche et dilue la cible de politique*. Sans la barrière qualité, un gain de 13 à 25 % aurait été adopté au prix de deux points de résolution. Au passage, le facteur ×8 de tentatives double les collisions sans gagner grand-chose.

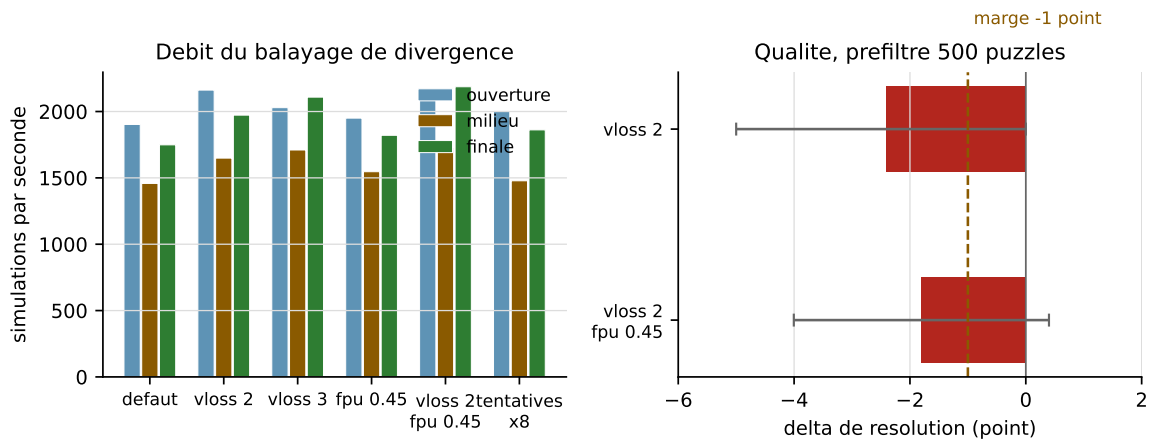


FIGURE 4 – À gauche, le débit du balayage. À droite, le delta de résolution et son intervalle bootstrap des deux candidats contre les défauts, sur 500 puzzles.

TABLEAU 10 – Verdict qualité, préfiltre 500 puzzles, lot fixe des deux côtés.

Candidat	Delta (point)	IC95	Discordantes (perdues/gagnées)	Verdict
vloss 2 + FPU 0,45	-1,8	[-4,0 ; +0,4]	21 / 12	echec
vloss 2	-2,4	[-5,0 ; 0,0]	27 / 15	echec

6.5 FP16

La conversion `convert_float_to_float16(keep_io_types=True)` produit un modèle de 7,3 Mio, lisible et correct, mais plus lent :

TABLEAU 11 – FP32 contre FP16, médianes par appel (ms), lots fixes.

Lot	FP32	FP16	Rapport
1	3,27 à 3,33	4,05 à 4,18	×1,22 à 1,28
8	3,45 à 3,52	4,24 à 4,31	×1,23
32	4,13 à 4,17	4,89 à 4,94	×1,18 à 1,19

Verdict sans campagne qualité : **echec**. Aucun code de production n’a été ajouté.

6.6 Le cas chaud

La figure 5 et le tableau 12 résument la partie auto-jouée avec arbre et pool persistants, 12 coups par position, 700 simulations par coup, tranches de 64, lot fixe.

Le résultat important n’est pas le débit, mais la colonne des appels réseau : **environ 700 appels pour 700 simulations, à chaque coup**, même quand les succès de table montent de 8 à 81 %. Un succès de table matérialise les enfants depuis la table et la descente continue vers une feuille réseau ; il n’évite donc pas l’inférence. La réserve Amdahl des rapports précédents, qui craignait que la réutilisation d’arbre ne réduise le gain, est sans objet. Seuls les terminaux évitent le réseau, et ils sont rares hors positions tactiques.

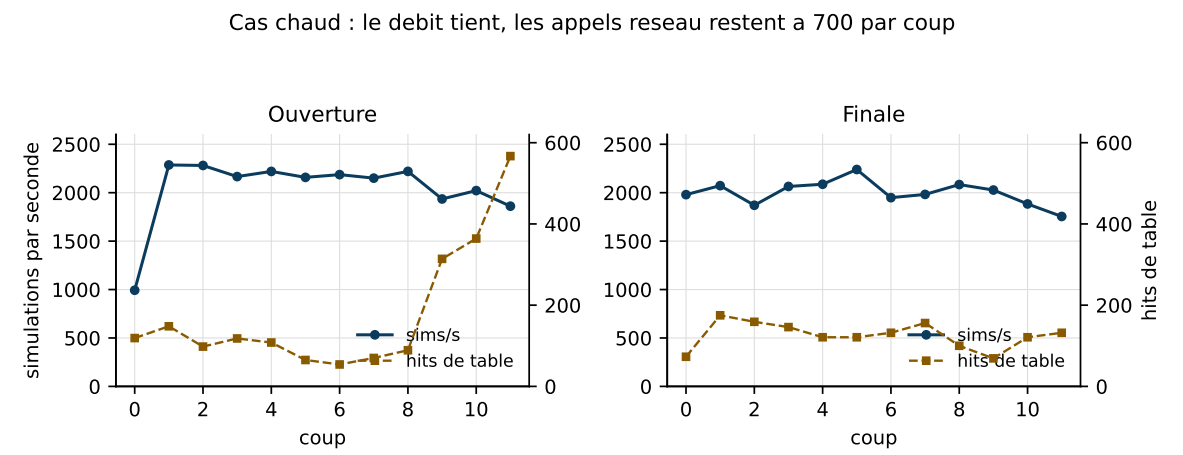


FIGURE 5 – Cas chaud, ouverture et finale : débit par coup et succès de table. Le premier coup paie l’initialisation de la session ONNX.

TABLEAU 12 – Cas chaud : médianes par position (le premier coup, qui paie l’initialisation de la session, est inclus mais visiblement plus lent).

Position	sims/s médian	appels réseau	succès de table
Ouverture	2 162	700 pour 700	54 à 567
Finale	2 005	700 pour 700	69 à 175

6.7 Synthèse

Piste	Effet mesuré	Verdict
Lot de forme fixe	×2,0 à ×3,2 de débit, p95 divisé	adopté
Buffers persistants	marginal, forme dominante	non retenu
Softmax	0,2 à 2 % de l’appel	non retenu
FP16	−18 à −28 % de débit	rejeté
Divergence	+13 à +25 % de débit, −1,8 à −2,4 points	rejeté

7 Discussion

7.1 Ce que le gain signifie

Le lot fixe ramène la recherche au niveau du self-play : environ 0,4 à 0,6 ms par position contre 0,38 ms pour le banc self-play à lot fixe 8, là où la recherche en payait 1,4 à 2,9. Les trois positions convergent vers 1 500 à 2 100 simulations par seconde, alors qu’elles s’étalonnaient de 350 à 850 auparavant. Le bot bénéficie directement de ce facteur, car l’évaluateur occupait 96 à 98 % du temps.

Le résultat dépasse l’estimation de l’étude de coût, qui annonçait ×3 à ×6 pour l’ensemble du chantier mais plaçait la première étape, la plomberie, à ×3 à ×5 au mieux : elle était juste, et elle est désormais acquise.

7.2 Le plafond restant

Le banc évaluateur montre que le coût par position continue de baisser jusqu’au lot 128 : 0,10 ms contre 0,63 ms au lot 8, soit un facteur six. Mais l’arbre d’un seul coup ne fournit que 5 à 7 feuilles par vague, et le padding évalue donc des doublons : au remplissage observé de 6,6, environ 17 % des rangées d’un lot sont des duplications. Deux directions restent :

1. **Remplir plus.** La divergence a montré qu'on ne remplit pas plus sans casser la qualité. Une file d'inférence continue, qui accumulerait les requêtes pendant le Run, ne forcerait rien et pourrait remplir 12 à 16 feuilles. Gain estimé : $\times 1,3$ à $\times 1,7$ sur l'évaluateur, donc sur la recherche.
2. **Descendre le coût par position.** Même à forme fixe, le GPU reste sous-occupé : 0,4 GFLOP par position pour 3,4 ms par appel de 8, soit quelques pour cent de la puissance crête. Le chemin passe par des noyaux mieux adaptés au 8×8 (TensorRT, fusion des couches), pas par la quantification, qui a empiré les choses.

Par rapport à Leela Chess Zero, l'écart s'est réduit sans disparaître. lc0 reste devant sur la taille de lot effective (32 à 256 contre 8), le backend et la file asynchrone. Le présent chantier a supprimé le facteur qui n'était ni algorithmique ni matériel : le contrat de forme imposé par ONNX Runtime.

7.3 Les leçons

Quatre leçons transférables

1. **Un contrat implicite peut coûter plus cher que l'algorithme.** Rien dans le code ne disait que la forme du lot devait être stable ; le runtime, lui, la re-planifiait à chaque changement.
2. **Mesurer hors de l'arbre.** Tant que l'appel est mesuré au milieu de la recherche, la cause reste noyée. Un banc d'API isolé a tranché en quelques minutes ce que trois hypothèses se disputaient depuis des semaines.
3. **Le protocole interleaved n'est pas un luxe.** La dérive entre sessions est du même ordre que les gains cherchés ; sans alternance, les conclusions auraient pu être inversées.
4. **La barrière qualité protège des victoires empoisonnées.** Le balayage de divergence offrait 13 à 25 % de débit pour 2 points de résolution ; seule la comparaison appariée pouvait le montrer.

7.4 Les limites

- Une seule machine et une seule carte ; les rapports absolus ne se transposent pas telles quelles, mais le mécanisme de re-planification est une propriété d'ONNX Runtime, pas du GPU.
- Le balayage de divergence repose sur 6 observations par position et le préfiltre qualité sur 500 puzzles ; une différence de moins d'un point n'aurait pas été distinguée du bruit.
- Le lot fixe évalue des doublons : à remplissage 6,6, environ 17 % des rangées sont perdues. C'est le prix de la forme stable, très inférieur au facteur quatre gagné.
- Le chemin mono est accéléré mais reste le fallback ; le self-play, qui évaluait déjà des lots fixes de 128 à 512, n'avait rien à corriger.
- Le bot utilise un modèle plus récent que celui des mesures ; le lot fixe ne change pas la recherche, donc la conclusion se transpose, mais les débits absolus diffèrent.

7.5 La production continue

Le verdict provisoire, établi dans le rapport de résultats, est le suivant : la production continue peut viser un remplissage de 12 à 16 feuilles et un gain de $\times 1,3$ à $\times 1,7$, pas davantage. Elle exige de synchroniser les statistiques lues pendant les backups, un coût et un risque de correction qui méritent un microbenchmark de file avant tout chantier. La recommandation

est d’écrire ce microbenchmark, puis un plan séparé avec les mêmes critères de médiane, de p95 et de qualité que les vagues.

8 Conclusion

Ce chantier a commencé par une anomalie : deux chemins qui partagent le même réseau, le même GPU et le même évaluateur, mais un facteur quatre de vitesse par position. Il se termine par une correction de quelques lignes, sémantiquement neutre, qui ramène la recherche UCI au niveau du self-play.

La démarche compte autant que le résultat. Un banc hors arbre a mesuré l’évaluateur seul, avec un mode qui reproduit fidèlement la façon dont la recherche alterne les tailles de lot. Ce mode a révélé que `session->Run` passe de 3 à 13 ms selon que la forme du lot est stable ou non. Le reste en découle : `padder` les lots à une forme fixe, ignorer les sorties dupliquées, et laisser la racine au lot 1.

Les mesures interleaved donnent $\times 2,0$ en ouverture, $\times 3,2$ en milieu et $\times 2,55$ en finale pour huit workers, avec un p95 de latence divisé par deux à trois. Le même gain s’applique au chemin mono, donc au banc de puzzles et aux tournois. Le bot UCI active le réglage.

Deux pistes ont été refusées sur preuves : FP16, plus lent de 18 à 28 %, et les réglages de divergence, plus rapides de 13 à 25 % mais moins précis de 1,8 à 2,4 points. Cette seconde réfutation est aussi instructive que le gain : elle montre que la barrière qualité du projet fonctionne et qu’un débit plus élevé n’est pas une fin en soi.

Enfin, la mesure du cas chaud a corrigé une erreur de raisonnement des rapports précédents : un succès de table n’évite pas l’inférence, car la descente continue vers une feuille réseau. La réutilisation d’arbre approfondit la recherche, elle n’allège pas le travail GPU, et la réserve Amdahl est levée.

Il reste du chemin vers Leela Chess Zero : une file d’inférence continue pour remplir des lots plus gros ($\times 1,3$ à $\times 1,7$), et un backend mieux adapté aux cartes 8×8. Le prochain jalon naturel est un microbenchmark de file, puis un plan séparé pour la production continue.

A Chronologie des commits

Le chantier occupe la journée du 19 septembre 2026, sur la branche `codex/cout-calcul-optimisations`, fusionnée en `fast-forward` dans `main`.

Commit	Message	Objet
07b9755	Complete l etude du cout	mise à jour de l’étude avec le régime réel du bot et la comparaison lc0
4cd91a4	Planifie les optimisations	plan d’implémentation en sept tâches
4161cd5	Mesure la courbe du lot	banc évaluateur hors arbre et timings internes
29f9458	Padde les lots d evaluation	lot de forme fixe dans les vagues, binding et options de banc
74122ac	Active le lot fixe dans l UCI	MCTS_FIXED_BATCH, test de construction
6606d97	Padde aussi les lots du chemin mono	mcts_search, banc de puzzles et tournois
4461d80	Consigne la cause du decrochage	section 6.2 de la spec, mise à jour du plafond
3515f00	Rend la divergence parametrisable	SearchTuning, garde de réservation, options de bancs
0c41d6f	Consigne le balayage et son rejet	tableau du balayage et verdict qualité
aed5f1a	Mesure le cas chaud	hot_tree_bench.py et son test
84bae7e	Documente les resultats	rejet FP16, correction Amdahl, rapport de résultats

Commit	Message	Objet
fb2ca71	Cloture les taches	statuts de la spec et du plan

Repères. Le banc évaluateur a été mesuré en trois campagnes (lots fixes 1 à 256, lots partiels, lots alternés) puis un A/B interleaved serré. Le lot fixe a été mesuré en A/B sur deux rounds pour huit workers et deux rounds pour le chemin mono. Le balayage de divergence a couvert neuf configurations, puis deux campagnes qualité de 500 puzzles. Le cas chaud a été mesuré sur deux positions et douze coups.

B Inventaire des fichiers

Fichier	Rôle
src/onnx_evaluator.hpp/.cpp	Contrat d'évaluation, appel ONNX, softmax, timings internes optionnels
src/mcts.hpp/.cpp	SearchTuning, set_fixed_batch, validation des réglages, FPU, réservation
src/mcts_wave.cpp	Padding des vagues, FPU, budget de tentatives
src/mcts_batch.cpp	Padding du chemin mono, vérification des dimensions de sortie
src/mcts_reservation.hpp/.cpp	Réservation en unités de virtual loss
src/bindings.cpp	set_fixed_batch, set_tuning, get_tuning
python_src/uci.py	MCTS_FIXED_BATCH, garde du chemin de production, journal de démarrage
python_src/search_bench.py	Lots fixes, divergence, tranches, collisions dans les mesures
python_src/puzzle_bench.py	Options de divergence du banc qualité, lot fixe par défaut
python_src/hot_tree_bench.py	Banc du cas chaud, agrégation pure
tests/cpp/evaluator_batch_bench.cpp	Banc hors arbre, lots fixes et alternés, timings
tests/cpp/test_wave_search.cpp	Padding vagues et mono, réglages de divergence
python_src/tests/test_search_bench.py	Propagation du lot fixe, de la divergence et des tranches
python_src/tests/test_bench_engine.py	Propagation côté banc de puzzles
python_src/tests/test_uci_race.py	Activation du lot fixe dans l'UCI
python_src/tests/test_hot_tree_bench.py	Agrégation du cas chaud
docs/superpowers/specs/2026-09-19-cout-calcul-cpu-gpu.md	Étude de coût, cause confirmée en 6.2, cas chaud en 8.4
docs/superpowers/plans/2026-09-19-cout-calcul-optimisations.md	Plan exécuté et statut
docs/superpowers/specs/2026-09-19-cout-calcul-resultats.md	Rapport de résultats et verdicts
docs/superpowers/specs/2026-09-16-multicore-waves-results.md	Correction de la réserve Amdahl

C Reproduction des mesures

C.1 Compilation et suites

```

1 cmake -S . -B build
2 cmake --build build --config Release
3 ctest --test-dir build -C Release --output-on-failure
4 uv run pytest python_src/tests -q

```

Listing 5 – Depuis la racine du dépôt.

C.2 Banc évaluateur

```

1 build/Release/evaluator_batch_bench.exe \
2   --model python_src/checkpoints_onnx/2026_04_23_23h25_iter316_unsupervised.onnx \
3   --gpu --batches 1 2 4 8 16 32 64 128 256 --repetitions 30 --rounds 20 \
4   > out/multicore/evaluator_batch_gpu.json
5
6 build/Release/evaluator_batch_bench.exe --model <modele> --gpu \
7   --variable --repetitions 30 --rounds 20 \
8   > out/multicore/evaluator_variable_gpu.json

```

Listing 6 – Courbe du lot et A/B alternance.

C.3 Lot fixe sur la recherche

```

1 uv run python python_src/search_bench.py --model <modele> --gpu \
2   --simulations 700 --passages 2 --repetitions 6 --batch-sizes 8 \
3   --worker-counts 8 --cache-history-depths 0 --tt-size 8192 \
4   --warmup-pool --out-json out/multicore/fixed-off-r1.json
5
6 uv run python python_src/search_bench.py --model <modele> --gpu \
7   --simulations 700 --passages 2 --repetitions 6 --batch-sizes 8 \
8   --worker-counts 8 --cache-history-depths 0 --tt-size 8192 \
9   --warmup-pool --fixed-batch --out-json out/multicore/fixed-on-r1.json

```

Listing 7 – A/B interleaved, 8 workers puis mono.

Répéter l’alternance pour deux rounds, puis refaire la paire avec un seul worker pour le chemin mono.

C.4 Balayage de divergence et qualité

```

1 uv run python python_src/search_bench.py --model <modele> --gpu \
2   --simulations 700 --slices 64 --passages 2 --repetitions 3 \
3   --batch-sizes 8 --worker-counts 8 --cache-history-depths 0 \
4   --tt-size 8192 --warmup-pool --fixed-batch \
5   --virtual-loss 2 --fpu 0.45 \
6   --out-json out/multicore/sweep-vl2fpu45.json
7
8 uv run python python_src/puzzle_bench.py --model <modele> \
9   --banc data/puzzles_bench.txt --gpu --travailleurs 1 --search-workers 8 \
10  --batch-size 8 --cache-history-depth 0 --simulations 700 --limite 500 \
11  --virtual-loss 2 --fpu 0.45 \
12  --out-csv out/multicore/tune-vl2fpu45.csv
13
14 uv run python python_src/multicore_comparison.py \
15  --baseline out/multicore/tune-base.csv \
16  --candidate out/multicore/tune-vl2fpu45.csv \
17  --out-report out/multicore/tune-comparison.md --seed 42

```

Listing 8 – Balayage, tranches de 64.

C.5 FP16 et cas chaud

```

1 uv run python -c "
2 import onnx
3 from onnxruntime.transformers.float16 import convert_float_to_float16
4 m = onnx.load('python_src/checkpoints_onnx/<modele>.onnx')
5 onnx.save(convert_float_to_float16(m, keep_io_types=True),
6           'python_src/checkpoints_onnx/<modele>.fp16.onnx')
7 "
8
9 uv run python python_src/hot_tree_bench.py --model <modele> --gpu \
10 --moves 12 --simulations 700 --slices 64 \
11 --out-json out/multicore/hot-tree.json

```

Listing 9 – Conversion et cas chaud.

C.6 Figures et rapport

```

1 uv run --with matplotlib python figures/generer_figures.py
2 pdflatex rapport.tex
3 pdflatex rapport.tex

```

Listing 10 – Depuis le dossier du rapport.

D Glossaire

Appel (inference call)

Un passage session->Run avec un lot de positions, soit une poignée de noyaux sur le GPU, soit une évaluation CPU.

Batch, lot

Ensemble de positions évaluées dans un seul appel réseau. Dans la recherche, c'est aussi la largeur d'une vague de collecte.

Fill, remplissage

Nombre de positions réelles par appel, mesuré par `nn_calls` / `nn_batches`. À lot demandé de 8, il tourne autour de 5 à 7 selon la position.

Fixed batch

Mode où chaque appel de vague est paddé à `batch_size` avec des rangées dupliquées, de sorties ignorées.

FP16

Format demi-précision. La conversion du modèle n'a pas accéléré ce réseau dans ce runtime.

FPU

First Play Urgency : valeur attribuée aux enfants non visités, ici parent moins une réduction proportionnelle à la masse de politique visitée. Le coefficient est un réglage de recherche.

NNCache, TT

Table des évaluations passées, indexée par la clé sémantique `h0`. Un succès matérialise les enfants, mais la descente continue : il n'évite pas l'inférence.

ONNX Runtime

Moteur d'exécution du réseau exporté, avec le provider CUDA sur ce projet.

Padding

Duplication du dernier tenseur pour garder une forme de lot constante, avec sorties ignorées.

PUCT

Règle de sélection de l'arbre, combinant exploitation, exploration et prior réseau, plus le virtual loss.

Self-play

Génération de parties par le moteur contre lui-même, avec des centaines de parties indépendantes et des lots de 128 à 512.

Session

Objet `Ort::Session` qui encapsule le graphe et le plan mémoire. C'est lui qui re-planifie quand la forme change.

Slices, tranches

Découpage d'une recherche en appels de 64 simulations, pour garder la main sur l'horloge UCI.

Tranche (slice)

Voir slice.

Virtual loss

Compteur `n_in_flight` incrémenté le long du chemin d'une descente, pour éviter que plusieurs descentes collectent la même feuille. Il n'entre pas dans Q .

Vague

Cycle complet : collecte parallèle d'un lot, barrière, inférence unique, expansion, remontées.

Références

- [1] D. Silver et al. *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*. arXiv :1712.01815, 2017.
- [2] LapZero-Chess. *Repartition du cout de calcul : CPU, GPU, et pistes d'optimisation*. docs/superpowers/specs/2026-09-19-cout-calcul-cpu-gpu.md, 2026.
- [3] LapZero-Chess. *Cout de calcul de la recherche : plan d'implementation*. docs/superpowers/plans/2026-09-19-cout-calcul-optimisations.md, 2026.
- [4] LapZero-Chess. *Cout de calcul de la recherche : resultats de l'assainissement*. docs/superpowers/specs/2026-09-19-cout-calcul-resultats.md, 2026.
- [5] LapZero-Chess. *Recherche MCTS multicoeur par vagues : mesures et decision*. docs/superpowers/specs/2026-09-16-multicore-waves-results.md, 2026.
- [6] LapZero-Chess. *Recherche MCTS multicœur par vagues : rapport d'ingenierie*. docs/rapports/2026-09-multicoeur/rapport.pdf, 2026.
- [7] Microsoft. *ONNX Runtime : documentation et sources*, version 1.24.3. <https://onnxruntime.ai>.
- [8] Leela Chess Zero. *Sources du moteur, branche master*. <https://github.com/LeelaChessZero/lc0>.
- [9] KataGo. *Sources du moteur*. <https://github.com/lightvector/KataGo>.
- [10] G. Amdahl. *Validity of the single processor approach to achieving large scale computing capabilities*. AFIPS, 1967.
- [11] IEEE. *IEEE Standard for Floating-Point Arithmetic (IEEE 754)*, 2019.